

Securitatea Sistemelor Informatic



- Curs 7.2 - Prezumpții criptografice dificile

Adela Georgescu

Facultatea de Matematică și Informatică
Universitatea din București
Anul universitar 2025-2026, semestrul II

Prezumptii criptografice dificile

- ▶ Criptografia modernă se bazează pe prezumpția că *anumite* probleme nu pot fi rezolvate în timp polinomial;

Prezumții criptografice dificile

- ▶ Criptografia modernă se bazează pe presupunerea că *anumite* probleme nu pot fi rezolvate în timp polinomial;
- ▶ Până acum am văzut că schemele de criptare și de autentificare se bazează pe presupunerea existenței permutărilor pseudoaleatoare;

Prezumții criptografice dificile

- ▶ Criptografia modernă se bazează pe presupunerea că *anumite* probleme nu pot fi rezolvate în timp polinomial;
- ▶ Până acum am văzut că schemele de criptare și de autentificare se bazează pe presupunerea existenței permutărilor pseudoaleatoare;
- ▶ Dar această presupunere e nenaturală și foarte puternică;

Prezumții criptografice dificile

- ▶ Criptografia modernă se bazează pe presupunerea că *anumite* probleme nu pot fi rezolvate în timp polinomial;
- ▶ Până acum am văzut că schemele de criptare și de autentificare se bazează pe presupunerea existenței permutărilor pseudoaleatoare;
- ▶ Dar această presupunere e nenaturală și foarte puternică;
- ▶ În practică, PRF pot fi instanțiate cu cifruri bloc;

Prezumții criptografice dificile

- ▶ Criptografia modernă se bazează pe presupunerea că *anumite* probleme nu pot fi rezolvate în timp polinomial;
- ▶ Până acum am văzut că schemele de criptare și de autentificare se bazează pe presupunerea existenței permutărilor pseudoaleatoare;
- ▶ Dar această presupunere e nenaturală și foarte puternică;
- ▶ În practică, PRF pot fi instanțiate cu cifruri bloc;
- ▶ Însă metode pentru a demonstra pseudoaleatorismul construcțiilor practice relativ la alte presupuneri "mai rezonabile" nu se cunosc;

Prezumții criptografice dificile

- ▶ Criptografia modernă se bazează pe presupunerea că *anumite* probleme nu pot fi rezolvate în timp polinomial;
- ▶ Până acum am văzut că schemele de criptare și de autentificare se bazează pe presupunerea existenței permutărilor pseudoaleatoare;
- ▶ Dar această presupunere e nenaturală și foarte puternică;
- ▶ În practică, PRF pot fi instanțiate cu cifruri bloc;
- ▶ Însă metode pentru a demonstra pseudoaleatorismul construcțiilor practice relativ la alte presupuneri "mai rezonabile" nu se cunosc;
- ▶ Dar e posibil a demonstra existența permutărilor pseudoaleatoare pe baza unei presupuneri mult mai slabe, cea a existenței funcțiilor one-way;

Prezumții criptografice dificile

- ▶ În continuare vom introduce câteva două considerate "dificile": problema factorizării și problema logaritmului discret și vom prezenta funcții conjecturate ca fiind one-way bazate pe aceste probleme;

Prezumții criptografice dificile

- ▶ În continuare vom introduce câteva două considerate "dificile": **problema factorizării** și **problema logaritmului discret** și vom prezenta funcții conjecturate ca fiind one-way bazate pe aceste probleme;
- ▶ Tot materialul ce urmează se bazează pe noțiuni de teoria numerelor;

Prezumții criptografice dificile

- ▶ În continuare vom introduce câteva două considerate "dificile": **problema factorizării** și **problema logaritmului discret** și vom prezenta funcții conjecturate ca fiind one-way bazate pe aceste probleme;
- ▶ Tot materialul ce urmează se bazează pe noțiuni de teoria numerelor;
- ▶ La criptografia *simetrică* (cu cheie secretă) am văzut primitive criptografice (i.e. funcții hash, PRG, PRF) care pot fi construite eficient fără a implica teoria numerelor;

Prezumpții criptografice dificile

- ▶ În continuare vom introduce câteva două considerate "dificile": **problema factorizării** și **problema logaritmului discret** și vom prezenta funcții conjecturate ca fiind one-way bazate pe aceste probleme;
- ▶ Tot materialul ce urmează se bazează pe noțiuni de teoria numerelor;
- ▶ La criptografia *simetrică* (cu cheie secretă) am văzut primitive criptografice (i.e. funcții hash, PRG, PRF) care pot fi construite eficient fără a implica teoria numerelor;
- ▶ La criptografia *asimetrică* (cu cheie publică) construcțiile cunoscute se bazează pe probleme matematice dificile din teoria numerelor;

1. Problema factorizării

- ▶ O primă problemă conjecturată ca fiind dificilă este problema factorizării numerelor întregi sau mai simplu problema factorizării;

1. Problema factorizării

- ▶ O primă problemă conjecturată ca fiind dificilă este **problema factorizării numerelor întregi** sau mai simplu **problema factorizării**;
- ▶ Fiind dat un număr compus N , problema cere să se găsească două numere prime x_1 și x_2 pe n biți așa încât $N = x_1 \cdot x_2$;

1. Problema factorizării

- ▶ O primă problemă conjecturată ca fiind dificilă este **problema factorizării numerelor întregi** sau mai simplu **problema factorizării**;
- ▶ Fiind dat un număr compus N , problema cere să se găsească două numere prime x_1 și x_2 pe n biți așa încât $N = x_1 \cdot x_2$;
- ▶ Cele mai greu de factorizat sunt numerele care au factori primi foarte mari.

Primalitate și factorizare

"The problem of distinguishing prime numbers from composite numbers and of resolving the latter into their prime factors is known to be one of the most important and useful in arithmetic. It has engaged the industry and wisdom of ancient and modern geometers to such an extent that it would be superfluous to discuss the problem at length... The dignity of the science itself seems to require that every possible means be explored for the solution of a problem so elegant and so celebrated."

(C.F.Gauss 1777 – 1855)

Generarea numerelor prime

- ▶ Pentru a putea folosi problema în criptografie, trebuie să generăm numere prime aleatoare *în mod eficient*;

Generarea numerelor prime

- ▶ Pentru a putea folosi problema în criptografie, trebuie să generăm numere prime aleatoare *în mod eficient*;
- ▶ Putem genera un număr prim aleator pe n biți prin alegerea repetată de numere aleatoare pe n biți până când găsim unul prim;

Generarea numerelor prime

- ▶ Pentru a putea folosi problema în criptografie, trebuie să generăm numere prime aleatoare *în mod eficient*;
- ▶ Putem genera un număr prim aleator pe n biți prin alegerea repetată de numere aleatoare pe n biți până când găsim unul prim;
- ▶ Pentru eficiență, ne interesează două aspecte:

Generarea numerelor prime

- ▶ Pentru a putea folosi problema în criptografie, trebuie să generăm numere prime aleatoare *în mod eficient*;
- ▶ Putem genera un număr prim aleator pe n biți prin alegerea repetată de numere aleatoare pe n biți până când găsim unul prim;
- ▶ Pentru eficiență, ne interesează două aspecte:
 1. probabilitatea ca un număr aleator de n biți să fie prim;

Generarea numerelor prime

- ▶ Pentru a putea folosi problema în criptografie, trebuie să generăm numere prime aleatoare *în mod eficient*;
- ▶ Putem genera un număr prim aleator pe n biți prin alegerea repetată de numere aleatoare pe n biți până când găsim unul prim;
- ▶ Pentru eficiență, ne interesează două aspecte:
 1. probabilitatea ca un număr aleator de n biți să fie prim;
 2. cum testăm eficient că un număr dat p este prim.

Generarea numerelor prime

- Pentru distribuția numerelor prime, se cunoaște următorul rezultat matematic:

Teoremă

Pentru orice $n > 1$, proporția de numere prime pe n biți este de cel puțin $1/3n$.

Generarea numerelor prime

- ▶ Pentru distribuția numerelor prime, se cunoaște următorul rezultat matematic:

Teoremă

Pentru orice $n > 1$, proporția de numere prime pe n biți este de cel puțin $1/3n$.

- ▶ Rezultă imediat că dacă testăm $t = 3n^2$ numere, probabilitatea ca un număr prim să nu fie ales este cel mult e^{-n} , deci neglijabilă.

Generarea numerelor prime

- ▶ Pentru distribuția numerelor prime, se cunoaște următorul rezultat matematic:

Teoremă

Pentru orice $n > 1$, proporția de numere prime pe n biți este de cel puțin $1/3n$.

- ▶ Rezultă imediat că dacă testăm $t = 3n^2$ numere, probabilitatea ca un număr prim să nu fie ales este cel mult e^{-n} , deci neglijabilă.
- ▶ Deci avem un algoritm în timp polinomial pentru găsirea numerelor prime

Testarea primalității

- ▶ Cei mai eficienți algoritmi sunt probabiliști:

Testarea primalității

- ▶ Cei mai eficienți algoritmi sunt probabiliști:
 - ▶ Dacă numărul p dat este prim atunci algoritmul întotdeauna întoarce rezultatul *prim*;

Testarea primalității

- ▶ Cei mai eficienți algoritmi sunt probabiliști:
 - ▶ Dacă numărul p dat este prim atunci algoritmul întotdeauna întoarce rezultatul *prim*;
 - ▶ Dacă p este compus, atunci cu probabilitate mare algoritmul va întoarce *compus*;

Testarea primalității

- ▶ Cei mai eficienți algoritmi sunt probabiliști:
 - ▶ Dacă numărul p dat este prim atunci algoritmul întotdeauna întoarce rezultatul *prim*;
 - ▶ Dacă p este compus, atunci cu probabilitate mare algoritmul va întoarce *compus*;
 - ▶ **Concluzie:** dacă outputul este *compus*, atunci p sigur este compus, dacă outputul este *prim*, atunci cu probabilitate mare p este prim dar este posibil și să se fi produs o eroare;

Testarea primalității

- ▶ Cei mai eficienți algoritmi sunt probabiliști:
 - ▶ Dacă numărul p dat este prim atunci algoritmul întotdeauna întoarce rezultatul *prim*;
 - ▶ Dacă p este compus, atunci cu probabilitate mare algoritmul va întoarce *compus*;
 - ▶ **Concluzie:** dacă outputul este *compus*, atunci p sigur este compus, dacă outputul este *prim*, atunci cu probabilitate mare p este prim dar este posibil și să se fi produs o eroare;
- ▶ Un **algoritm determinist polinomial** a fost propus în 2002, dar este mai lent decât algoritmii probabiliști;

Testarea primalității

- ▶ Cei mai eficienți algoritmi sunt probabiliști:
 - ▶ Dacă numărul p dat este prim atunci algoritmul întotdeauna întoarce rezultatul *prim*;
 - ▶ Dacă p este compus, atunci cu probabilitate mare algoritmul va întoarce *compus*;
 - ▶ **Concluzie:** dacă outputul este *compus*, atunci p sigur este compus, dacă outputul este *prim*, atunci cu probabilitate mare p este prim dar este posibil și să se fi produs o eroare;
- ▶ Un **algoritm determinist polinomial** a fost propus în 2002, dar este mai lent decât algoritmii probabiliști;
- ▶ Un **algoritm probabilist** foarte răspândit este Miller-Rabin care acceptă la intrare un număr N și rulează în timp polinomial în $|N|$

Testarea primalității

- ▶ **Rezultat:** Dacă p este prim, atunci alg. Miller-Rabin întoarce întotdeauna "prim". Dacă însă p este compus, atunci algoritmul întoarce "prim" cu probabilitate neglijabilă cel mult 2^{-t} .
- ▶ Acest algoritm se bazează pe o proprietate pe care numerele prime o au.
- ▶ Dacă N este prim atunci $|\mathbb{Z}_N^*| = N - 1$ și atunci pentru orice $a \in \{1, \dots, N - 1\}$ avem ca $a^{N-1} = 1 \pmod{N}$.

Algoritmi de factorizare

- ▶ Deocamdată nu se cunosc *algoritmi polinomiali* pentru problema factorizării;

Algoritmi de factorizare

- ▶ Deocamdată nu se cunosc *algoritmi polinomiali* pentru problema factorizării;
- ▶ Dar există algoritmi mult mai buni decât forța brută;

Algoritmi de factorizare

- ▶ Deocamdată nu se cunosc *algoritmi polinomiali* pentru problema factorizării;
- ▶ Dar există algoritmi mult mai buni decât forța brută;
- ▶ Prezentăm în continuare câțiva algoritmi de factorizare.

Algoritmi de factorizare

- **Reamintim:** Fiind dat un număr compus N , **problema factorizării** cere să se găsească 2 numere prime p și q a.î.
 $N = pq$;

Algoritmi de factorizare

- ▶ **Reamintim:** Fiind dat un număr compus N , **problema factorizării** cere să se găsească 2 numere prime p și q a.î. $N = pq$;
- ▶ Considerăm $|p| = |q| = n$ și deci $n = O(\log N)$;

Algoritmi de factorizare

- ▶ **Reamintim:** Fiind dat un număr compus N , **problema factorizării** cere să se găsească 2 numere prime p și q a.î. $N = pq$;
- ▶ Considerăm $|p| = |q| = n$ și deci $n = O(\log N)$;
- ▶ Metoda cea mai simplă este împărțirea numărului N prin toate numerele p impare din intervalul $p = 3, \dots, \lfloor \sqrt{N} \rfloor$.

Algoritmi de factorizare

- ▶ **Reamintim:** Fiind dat un număr compus N , **problema factorizării** cere să se găsească 2 numere prime p și q a.î. $N = pq$;
- ▶ Considerăm $|p| = |q| = n$ și deci $n = O(\log N)$;
- ▶ Metoda cea mai simplă este împărțirea numărului N prin toate numerele p impare din intervalul $p = 3, \dots, \lfloor \sqrt{N} \rfloor$.
- ▶ Complexitatea timp este $O(\sqrt{N} \cdot (\log N)^c)$ unde c este o constantă, adică exponentială în numărul b de biți al lui N ; $b = \log_2 N$. Complexitatea este $O(N^{1/2}) = O((2^{\log_2 N})^{1/2})$

Algoritmi de factorizare

- ▶ **Reamintim:** Fiind dat un număr compus N , **problema factorizării** cere să se găsească 2 numere prime p și q a.î. $N = pq$;
- ▶ Considerăm $|p| = |q| = n$ și deci $n = O(\log N)$;
- ▶ Metoda cea mai simplă este împărțirea numărului N prin toate numerele p impare din intervalul $p = 3, \dots, \lfloor \sqrt{N} \rfloor$.
- ▶ Complexitatea timp este $O(\sqrt{N} \cdot (\log N)^c)$ unde c este o constantă, adică exponentială în numărul b de biți al lui N ; $b = \log_2 N$. Complexitatea este $O(N^{1/2}) = O((2^{\log_2 N})^{1/2})$
- ▶ Pentru $N < 10^{12}$ metoda este destul de eficientă.

Algoritmi de factorizare

- ▶ Există însă algoritmi mai sofisticăți, cu timp de execuție mai bun, între care:

Algoritmi de factorizare

- ▶ Există însă algoritmi mai sofisticăți, cu timp de execuție mai bun, între care:
 - ▶ Metoda **Pollard $p - 1$** : funcționează atunci când $p - 1$ are factori primi "mici";

Algoritmi de factorizare

- ▶ Există însă algoritmi mai sofisticăți, cu timp de execuție mai bun, între care:
 - ▶ Metoda **Pollard p – 1**: funcționează atunci când $p - 1$ are factori primi "mici";
 - ▶ Metoda **Pollard rho**: timpul de execuție este $O(N^{1/4} \cdot (\log N)^c)$, deci tot exponențial în lungimea lui N ;

Algoritmi de factorizare

- ▶ Există însă algoritmi mai sofisticăți, cu timp de execuție mai bun, între care:
 - ▶ Metoda **Pollard $p - 1$** : funcționează atunci când $p - 1$ are factori primi "mici";
 - ▶ Metoda **Pollard rho**: timpul de execuție este $O(N^{1/4} \cdot (\log N)^c)$, deci tot exponențial în lungimea lui N ;
 - ▶ **Algoritmul sitei pătratice** - rulează în timp *sub-exponențial* în lungimea lui N .

Algoritmi de factorizare

- ▶ Există însă algoritmi mai sofisticăți, cu timp de execuție mai bun, între care:
 - ▶ Metoda **Pollard $p - 1$** : funcționează atunci când $p - 1$ are factori primi "mici";
 - ▶ Metoda **Pollard rho**: timpul de execuție este $O(N^{1/4} \cdot (\log N)^c)$, deci tot exponențial în lungimea lui N ;
 - ▶ **Algoritmul sitei pătratice** - rulează în timp *sub-exponențial* în lungimea lui N .
- ▶ Deocamdată, cel mai rapid algoritm de factorizare este o îmbunătățire a sitei pătratice care factorizează un număr N de lungime $O(n)$ în timp $2^{O(n^{1/3} \cdot (\log n)^{2/3})}$.

RSA Challenge

- ▶ În 1991, Laboratoarele RSA lansează *RSA Challenge*;

RSA Challenge

- ▶ În 1991, Laboratoarele RSA lansează *RSA Challenge*;
- ▶ Aceasta presupune factorizarea unor numere N , unde $N = p \cdot q$, cu p, q 2 numere prime mari;

RSA Challenge

- ▶ În 1991, Laboratoarele RSA lansează *RSA Challenge*;
- ▶ Aceasta presupune factorizarea unor numere N , unde $N = p \cdot q$, cu p, q 2 numere prime mari;
- ▶ Au fost lansate mai multe provocări, câte 1 pentru fiecare dimensiune (în biți) a lui N :

RSA Challenge

- ▶ În 1991, Laboratoarele RSA lansează *RSA Challenge*;
- ▶ Aceasta presupune factorizarea unor numere N , unde $N = p \cdot q$, cu p, q 2 numere prime mari;
- ▶ Au fost lansate mai multe provocări, câte 1 pentru fiecare dimensiune (în biți) a lui N :
- ▶ Exemple includ: RSA-576, RSA-640, RSA-768, $\dots$, RSA-1024, RSA-1536, RSA-2048;

RSA Challenge

- ▶ În 1991, Laboratoarele RSA lansează *RSA Challenge*;
- ▶ Aceasta presupune factorizarea unor numere N , unde $N = p \cdot q$, cu p, q 2 numere prime mari;
- ▶ Au fost lansate mai multe provocări, câte 1 pentru fiecare dimensiune (în biți) a lui N :
- ▶ Exemple includ: RSA-576, RSA-640, RSA-768, $\dots$, RSA-1024, RSA-1536, RSA-2048;
- ▶ Provocarea s-a încheiat oficial în 2007;

RSA Challenge

- ▶ În 1991, Laboratoarele RSA lansează *RSA Challenge*;
- ▶ Aceasta presupune factorizarea unor numere N , unde $N = p \cdot q$, cu p, q 2 numere prime mari;
- ▶ Au fost lansate mai multe provocări, câte 1 pentru fiecare dimensiune (în biți) a lui N :
- ▶ Exemple includ: RSA-576, RSA-640, RSA-768, $\dots$, RSA-1024, RSA-1536, RSA-2048;
- ▶ Provocarea s-a încheiat oficial în 2007;
- ▶ Multe provocări au fost sparte în cursul anilor (chiar și ulterior închiderii oficiale), însă există numere încă nefactorizate:

RSA Challenge

► RSA-1024

13506641086599522334960321627880596993888147560566
70275244851438515265106048595338339402871505719094
41798207282164471551373680419703964191743046496589
27425623934102086438320211037295872576235850964311
05640735015081875106765946292055636855294752135008
52879416377328533906109750544334999811150056977236
890927563

RSA Challenge

► RSA-2048

25195908475657893494027183240048398571429282126204
03202777713783604366202070759555626401852588078440
69182906412495150821892985591491761845028084891200
72844992687392807287776735971418347270261896375014
97182469116507761337985909570009733045974880842840
17974291006424586918171951187461215151726546322822
16869987549182422433637259085141865462043576798423
38718477444792073993423658482382428119816381501067
48104516603773060562016196762561338441436038339044
14952634432190114657544454178424020924616515723350
77870774981712577246796292638635637328991215483143
81678998850404453640235273819513786365643912120103
97122822120720357

2. Problema logaritmului discret

- ▶ O altă prezumție dificilă este DLP (Discrete Logarithm Problem) (sau PLD (Problema Logaritmului Discret));

2. Problema logaritmului discret

- ▶ O altă prezumție dificilă este DLP (Discrete Logarithm Problem) (sau PLD (Problema Logaritmului Discret));
- ▶ Considerăm $\mathbb{G}$ un grup ciclic de ordin q ;

2. Problema logaritmului discret

- ▶ O altă prezumție dificilă este DLP (Discrete Logarithm Problem) (sau PLD (Problema Logaritmului Discret));
- ▶ Considerăm $\mathbb{G}$ un grup ciclic de ordin q ;
- ▶ Există un generator $g \in \mathbb{G}$ a.î. $\mathbb{G} = \{g^0, g^1, \dots, g^{q-1}\}$;

2. Problema logaritmului discret

- ▶ O altă prezumție dificilă este DLP (Discrete Logarithm Problem) (sau PLD (Problema Logaritmului Discret));
- ▶ Considerăm $\mathbb{G}$ un grup ciclic de ordin q ;
- ▶ Există un generator $g \in \mathbb{G}$ a.î. $\mathbb{G} = \{g^0, g^1, \dots, g^{q-1}\}$;
- ▶ Echivalent, pentru fiecare $h \in \mathbb{G}$ există un *unic* $x \in \mathbb{Z}_q$ a.î. $g^x = h$;

2. Problema logaritmului discret

- ▶ O altă prezumție dificilă este DLP (Discrete Logarithm Problem) (sau PLD (Problema Logaritmului Discret));
- ▶ Considerăm $\mathbb{G}$ un grup ciclic de ordin q ;
- ▶ Există un generator $g \in \mathbb{G}$ a.î. $\mathbb{G} = \{g^0, g^1, \dots, g^{q-1}\}$;
- ▶ Echivalent, pentru fiecare $h \in \mathbb{G}$ există un *unic* $x \in \mathbb{Z}_q$ a.î. $g^x = h$;
- ▶ x se numește **logaritmul discret** al lui h în raport cu g și se notează

$$x = \log_g h$$

Experimentul logaritmului discret $DLog_{\mathcal{A}}(n)$

1. Generează $(\mathbb{G}, q, g)$ unde $\mathbb{G}$ este un grup ciclic de ordin q (cu $|q| = n$) iar g este un generator al lui $\mathbb{G}$.

Experimentul logaritmului discret $DLog_{\mathcal{A}}(n)$

1. Generează $(\mathbb{G}, q, g)$ unde $\mathbb{G}$ este un grup ciclic de ordin q (cu $|q| = n$) iar g este un generator al lui $\mathbb{G}$.
2. Alege $h \leftarrow^R \mathbb{G}$. (se poate alege $x' \leftarrow^R \mathbb{Z}_q$ și apoi $h := g^{x'}$.)

Experimentul logaritmului discret $DLog_{\mathcal{A}}(n)$

1. Generează $(\mathbb{G}, q, g)$ unde $\mathbb{G}$ este un grup ciclic de ordin q (cu $|q| = n$) iar g este un generator al lui $\mathbb{G}$.
2. Alege $h \leftarrow^R \mathbb{G}$. (se poate alege $x' \leftarrow^R \mathbb{Z}_q$ și apoi $h := g^{x'}$.)
3. $\mathcal{A}$ primește $\mathbb{G}, q, g, h$ și întoarce $x \in \mathbb{Z}_q$;

Experimentul logaritmului discret $DLog_{\mathcal{A}}(n)$

1. Generează $(\mathbb{G}, q, g)$ unde $\mathbb{G}$ este un grup ciclic de ordin q (cu $|q| = n$) iar g este un generator al lui $\mathbb{G}$.
2. Alege $h \leftarrow^R \mathbb{G}$. (se poate alege $x' \leftarrow^R \mathbb{Z}_q$ și apoi $h := g^{x'}$.)
3. $\mathcal{A}$ primește $\mathbb{G}, q, g, h$ și întoarce $x \in \mathbb{Z}_q$;
4. Output-ul experimentului este 1 dacă $g^x = h$ și 0 altfel.

Experimentul logaritmului discret $DLog_{\mathcal{A}}(n)$

1. Generează $(\mathbb{G}, q, g)$ unde $\mathbb{G}$ este un grup ciclic de ordin q (cu $|q| = n$) iar g este un generator al lui $\mathbb{G}$.
2. Alege $h \leftarrow^R \mathbb{G}$. (se poate alege $x' \leftarrow^R \mathbb{Z}_q$ și apoi $h := g^{x'}$.)
3. $\mathcal{A}$ primește $\mathbb{G}, q, g, h$ și întoarce $x \in \mathbb{Z}_q$;
4. Output-ul experimentului este 1 dacă $g^x = h$ și 0 altfel.

Definiție

Spunem că problema logaritmului discret (DLP) este dificilă dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât

$$\Pr[DLog_{\mathcal{A}}(n) = 1] \leq \text{negl}(n)$$

Grupuri ciclice de ordin prim

- ▶ Există câteva clase de grupuri ciclice pentru care DLP este considerată dificilă;

Grupuri ciclice de ordin prim

- ▶ Există câteva clase de grupuri ciclice pentru care DLP este considerată dificilă;
- ▶ Una dintre ele este clasa grupurilor ciclice de *ordin prim* (în aceste grupuri, problema este "cea mai dificilă");

Grupuri ciclice de ordin prim

- ▶ Există câteva clase de grupuri ciclice pentru care DLP este considerată dificilă;
- ▶ Una dintre ele este clasa grupurilor ciclice de *ordin prim* (în aceste grupuri, problema este "cea mai dificilă");
- ▶ DLP nu poate fi rezolvată în timp polinomial în grupurile care nu sunt de ordin prim, ci doar este *mai ușoară*;

Grupuri ciclice de ordin prim

- ▶ Există câteva clase de grupuri ciclice pentru care DLP este considerată dificilă;
- ▶ Una dintre ele este clasa grupurilor ciclice de *ordin prim* (în aceste grupuri, problema este "cea mai dificilă");
- ▶ DLP nu poate fi rezolvată în timp polinomial în grupurile care nu sunt de ordin prim, ci doar este *mai ușoară*;
- ▶ În aceste grupuri căutarea unui generator și verificarea că un număr dat este generator sunt triviale.

Lucrul în $\mathbb{Z}_p^*$

- ▶ DLP este considerată dificilă în grupuri ciclice de forma $\mathbb{Z}_p^*$ cu p prim;

Lucrul în $\mathbb{Z}_p^*$

- ▶ DLP este considerată dificilă în grupuri ciclice de forma $\mathbb{Z}_p^*$ cu p prim;
- ▶ Însă pentru $p > 3$ grupul $\mathbb{Z}_p^*$ NU are ordin prim;

Lucrul în $\mathbb{Z}_p^*$

- ▶ DLP este considerată dificilă în grupuri ciclice de forma $\mathbb{Z}_p^*$ cu p prim;
- ▶ Însă pentru $p > 3$ grupul $\mathbb{Z}_p^*$ NU are ordin prim;
- ▶ Aceasta problemă se rezolvă folosind un *subgrup* potrivit al lui $\mathbb{Z}_p^*$;

Important de reținut!

- ▶ Cel mai rapid algoritm de factorizare necesită timp sub-exponențial;
- ▶ Problema logaritmului discret este dificilă.